

Numérique Durable (TI616)

Mini-Projet Hackathon — « Le site web le plus Green »

Rapport Final — Partie 1 + Partie 2

GreenCode Hub

La plateforme collaborative du Green Coding

URL du site : <https://greencode-hub.onrender.com>

Dépôt GitHub : <https://github.com/YvesDK07/GreenCode-Hub>

Équipe — Groupe AFR2

Yves de KERROS — Back-end (routes API, authentification)

Raphael GIRARD — Front-end (pages, composants, CSS)

Lestyn JONES — Base de données (modèles, requêtes, migrations)

Yohan DE LANDTSHEER — Documentation, tests, déploiement

Robinson DIALLO — Front-end complémentaire, wireframes, design sobre

Module TI616 — EFREI Paris — 2025/2026

Section 1 — Présentation du projet

Proposition de valeur

GreenCode Hub est un site sobre où les développeurs partagent et comparent des snippets de code classés par efficacité énergétique selon leur complexité algorithmique. L'objectif est double : sensibiliser à l'impact énergétique du code et offrir une référence pratique pour écrire du code plus sobre.

Périmètre MVP

Le MVP couvre le CRUD utilisateurs complet (inscription, connexion, profil, suppression), le CRUD snippets complet (création, affichage, modification, suppression), le système de vote (un seul par utilisateur par snippet), la navigation par catégorie avec filtres, le bloc d'engagements écologiques en page d'accueil, ainsi qu'un panneau d'administration permettant la gestion des utilisateurs (liste paginée, changement de rôle, suppression), la gestion des catégories (CRUD complet) et la modération de tout snippet.

Utilisateurs cibles

Étudiants en informatique sensibilisés au Green IT, développeurs professionnels souhaitant optimiser leur code, enseignants cherchant des exemples concrets d'impact énergétique du code.

Contraintes Green IT retenues

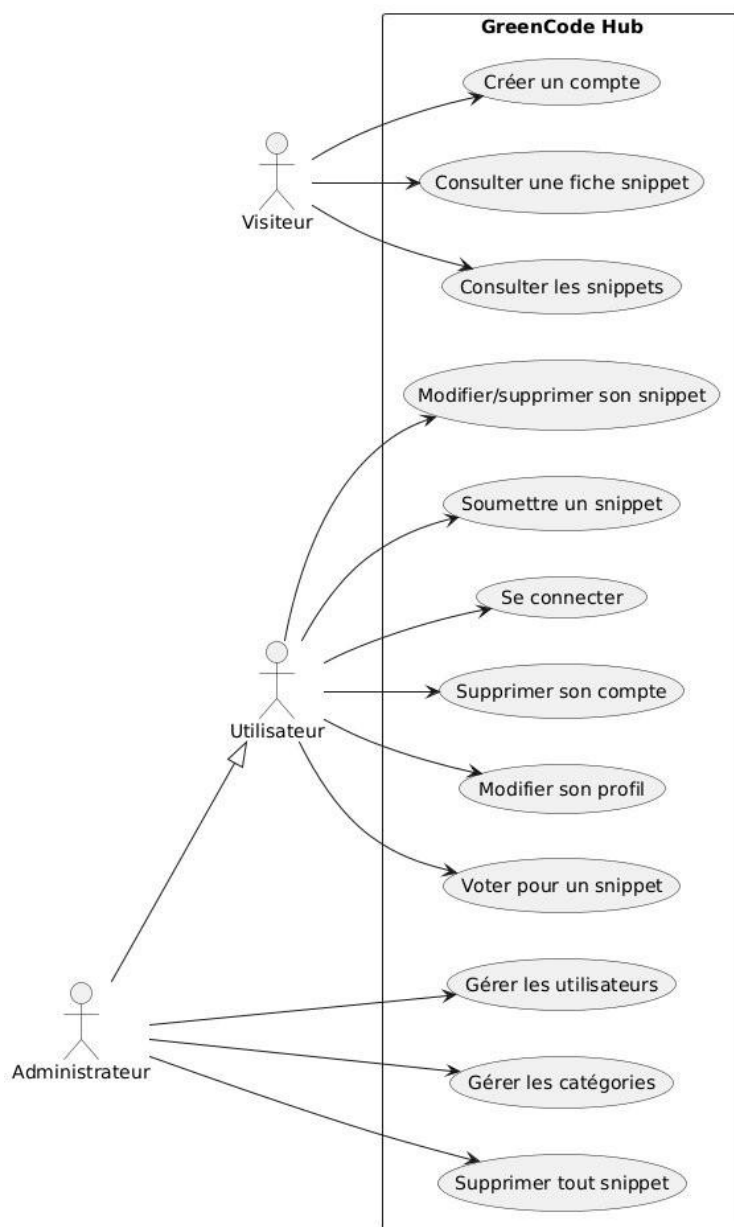
- Pages inférieures à 300 Ko (objectif largement atteint : pages de 4 Ko mesurées).
- Moins de 10 requêtes HTTP par page (objectif atteint : 2 requêtes mesurées).
- Zéro tracking, zéro cookie tiers, zéro image lourde.
- Polices système uniquement (font-family: system-ui).
- Rendu côté serveur (SSR) pour éviter un bundle JS lourd.
- 5 dépendances npm uniquement.
- Contenu 100 % textuel (le code source est par nature du texte pur).

Section 2 — Architecture et conception

Architecture générale

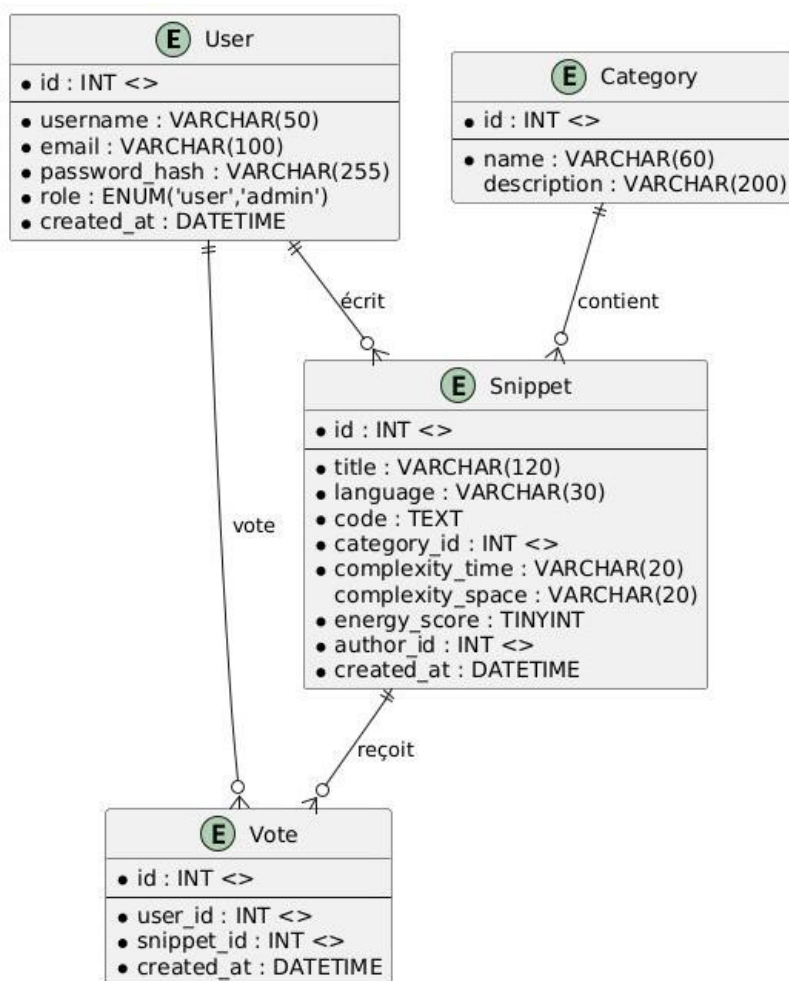
Le système suit une architecture trois tiers classique : une couche de présentation (HTML/CSS + EJS rendu côté serveur), une couche applicative (Node.js + Express), et une couche de données (SQLite via sql.js). Le serveur Express reçoit les requêtes HTTP, interroge la base SQLite via des requêtes préparées, génère le HTML côté serveur et le renvoie au navigateur. Cette approche SSR évite tout bundle JavaScript lourd côté client.

Diagramme de cas d'utilisation



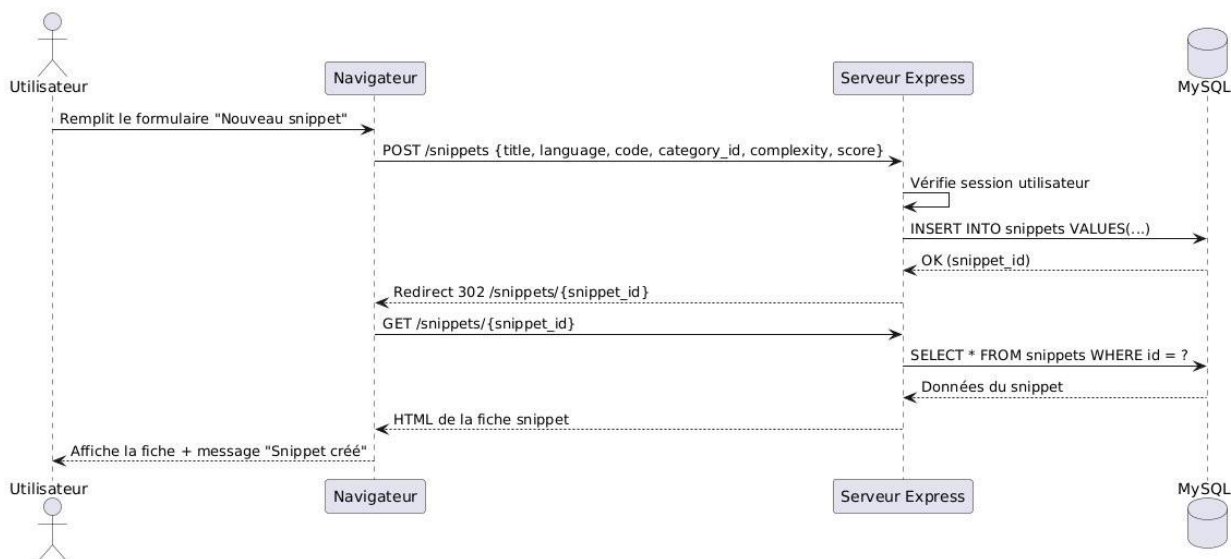
Note : L'Utilisateur hérite des cas du Visiteur (consulter snippets, consulter fiche). L'Administrateur hérite de l'Utilisateur et ajoute la gestion des utilisateurs, catégories et la suppression de tout snippet.

Schéma conceptuel des données



4 tables : users, snippets, categories, votes (contrainte UNIQUE user_id + snippet_id).

Diagramme de séquence — Soumission d'un snippet



L'utilisateur remplit le formulaire, le navigateur envoie un POST /new, le serveur vérifie la session, insère en BDD, et redirige vers la fiche du snippet créé.

Section 3 — Choix technologiques justifiés

Élément	Choix	Justification Green IT
Front-end	HTML/CSS natif + EJS (SSR)	Zéro framework front-end lourd. Le HTML est généré côté serveur, aucun bundle JS n'est envoyé au client. CSS unique de 8.4 Ko (6.2 Ko après minification). Polices système. Zéro image. Pages mesurées à 4 Ko.
Back-end	Node.js + Express.js	Framework minimaliste. Rendu SSR avec EJS. Routes clairement séparées (auth.js, snippets.js). Middleware léger. Gestion d'erreurs avec codes HTTP appropriés.
Base de données	SQLite (sql.js)	Zéro serveur BDD séparé = zéro consommation supplémentaire. Requêtes préparées (anti-injection SQL). Fichier unique. 4 tables, schéma normalisé avec clés étrangères et contraintes d'unicité. Index automatiques SQLite sur les PRIMARY KEY et UNIQUE.
Authentification	bcryptjs + express-session	Hashage bcrypt (coût 10). Sessions côté serveur (pas de JWT = pas de payload répété dans chaque requête). Pas de cookie tiers.
Déploiement	Render (free tier)	Infrastructure mutualisée. Déploiement automatique depuis GitHub (branche main). Pas de VPS dédié = empreinte partagée.
Outils projet	GitHub + Discord	GitHub centralise code, issues, PR, projects. Pas d'outil supplémentaire (Jira, Trello, Notion). Discord déjà utilisé par l'équipe.

Sobriété des dépendances

Dépendance	Taille (gzipé)	Rôle	Alternative écartée
express	~200 Ko	Framework HTTP minimaliste	Koa, Fastify (plus de dépendances)
ejs	~40 Ko	Moteur de template SSR	React/Vue SSR (bundle lourd)
sql.js	~800 Ko	SQLite en pur JS	MySQL/PostgreSQL (serveur séparé)
bcryptjs	~20 Ko	Hashage mots de passe	argon2 (dépendance native C++)
express-session	~15 Ko	Gestion sessions	JWT (payload répété par requête)

Total : 5 dépendances directes. Aucune bibliothèque superflue. Chaque choix est justifié par le besoin réel.

Section 4 — Implémentation

Structure du projet

```
GreenCode-Hub/
  server.js           # Point d'entrée Express (64 lignes)
  db/init.js          # Init BDD + seed (157 lignes)
  middleware/auth.js   # Middlewares auth (27 lignes)
  routes/auth.js       # CRUD utilisateurs (117 lignes)
  routes/snippets.js   # CRUD snippets + votes (136 lignes)
  routes/admin.js      # Routes administration (172 lignes)
  views/              # 13 templates EJS (+ 2 partials)
  public/style.css     # CSS unique (8,4 Ko)
  .github/workflows/   # ci.yml (CI/CD)
  docs/                # Rapport + captures Lighthouse / EcoIndex
  .gitignore
  .env.example
  README.md
```

Total : environ 670 lignes de code métier (hors node_modules), 13 templates EJS, 1 fichier CSS unique.

CRUD Utilisateurs

Création (POST /register) : validation côté serveur (champs obligatoires, email unique, mot de passe ≥ 6 caractères, confirmation). Hashage bcrypt. Création en BDD. Connexion automatique par session.

Lecture (GET /profile) : affichage du profil avec statistiques (snippets publiés, votes donnés). Liste des snippets de l'utilisateur avec liens modifier/supprimer.

Modification (POST /profile) : vérification du mot de passe actuel obligatoire. Modification username, email, mot de passe. Contrôle d'unicité.

Suppression (POST /profile/delete) : confirmation JavaScript obligatoire. Suppression des votes associés. Anonymisation des snippets (author_id = 0). Destruction de session.

CRUD Snippets

Création (POST /new) : formulaire complet (titre, langage, catégorie, code, complexité, score de sobriété). Validation serveur.

Lecture — Catalogue (GET /catalogue) : liste filtrée et triable (par catégorie, langage, tri par votes/score/récent). Affichage en grille de cartes avec badges (langage, complexité, score en étoiles, votes).

Lecture — Fiche (GET /snippet/:id) : affichage complet, bloc de code sur fond sombre, métriques visuelles, bouton voter (toggle), boutons modifier/supprimer (si auteur ou admin).

Modification (POST /snippet/:id/edit) : accessible uniquement à l'auteur ou à l'admin. Formulaire pré-rempli. Mise à jour en BDD.

Suppression (POST /snippet/:id/delete) : confirmation JS. Suppression des votes associés puis du snippet. Redirection catalogue.

Système de votes

Un utilisateur connecté peut voter une seule fois par snippet (contrainte UNIQUE user_id + snippet_id). Le vote fonctionne en toggle : cliquer une deuxième fois retire le vote. Le compteur est recalculé à chaque affichage via une sous-requête COUNT(*).

Panneau d'administration

Le panneau admin (routes/admin.js) regroupe les trois cas d'utilisation administrateur identifiés en Partie 1. L'accès est protégé par le middleware requireAdmin qui vérifie que la session courante porte bien le rôle admin (sinon HTTP 403). Le lien vers /admin n'apparaît dans la barre de navigation que si user.role === 'admin'.

Dashboard (GET /admin) : tableau de bord avec les statistiques globales (utilisateurs, admins, snippets, catégories, votes) et trois cartes d'action vers les sous-sections.

Gérer les utilisateurs (GET /admin/users) : liste paginée à 20 résultats par page (LIMIT/OFFSET), avec username, email, rôle, nombre de snippets, nombre de votes et date d'inscription. Chaque ligne propose deux actions : basculer le rôle (POST /admin/users/:id/role) et supprimer (POST /admin/users/:id/delete avec confirmation JS et anonymisation des snippets associés).

Gérer les catégories (GET /admin/categories) : CRUD complet avec création (POST /admin/categories), édition inline (POST /admin/categories/:id/edit) et suppression (POST /admin/categories/:id/delete). La suppression est bloquée si la catégorie contient encore des snippets, et la création/édition rejette les doublons (vérification insensible à la casse).

Modérer les snippets : déjà couvert par routes/snippets.js — un admin peut modifier ou supprimer n'importe quel snippet via les routes existantes /snippet/:id/edit et /snippet/:id/delete (la condition de propriété accepte aussi req.session.role === 'admin').

Verrous de sécurité : trois garde-fous protègent l'intégrité du système. (1) Un admin ne peut pas se rétrograder ni se supprimer lui-même (évite le logout). (2) Le dernier administrateur restant ne peut pas être rétrogradé ni supprimé (vérification du COUNT des admins avant action). (3) Une catégorie utilisée par au moins un snippet ne peut pas être supprimée. Toutes ces vérifications retournent un message d'erreur explicite à l'utilisateur.

Extrait de code commenté — Middleware d'authentification

```
// Vérifie que l'utilisateur est connecté
function requireAuth(req, res, next) {
  if (!req.session.userId) {
    return res.redirect('/login'); // Redirige si non connecté
  }
  next(); // Passe au handler suivant
}

// Injecte l'utilisateur dans toutes les vues EJS
function injectUser(db) {
  return (req, res, next) => {
    res.locals.user = null;
    if (req.session.userId) {
      res.locals.user = db.prepare(
        'SELECT id, username, email, role FROM users WHERE id = ?'
      ).get(req.session.userId);
    }
    next();
  };
}
```

```
    ).get(req.session.userId); // 1 seule requête par page
  }
  next();
};
}
```

Ce middleware illustre deux principes : sécurité (protection des routes) et sobriété (une seule requête BDD par page, `SELECT` ciblé sur 4 colonnes).

Choix d'éco-conception front-end appliqués

- **Architecture SSR (Server-Side Rendering)** : tout le HTML est généré côté serveur par EJS, aucun bundle JavaScript n'est envoyé au client. Comparé à une SPA React/Vue équivalente (typiquement 150 à 300 Ko de JS), le gain est massif.
- **CSS unique non-framework** : un seul fichier `style.css` de 8 581 octets ($\approx 8,4$ Ko), sans Tailwind ni Bootstrap, chargé sur toutes les pages. Pas de pré-processeur, pas de pipeline de build.
- **Polices système uniquement** : `font-family: system-ui, -apple-system, sans-serif`. Aucune requête vers Google Fonts ou un CDN externe.
- **Zéro image** : contenu 100 % textuel, le code source étant par nature du texte pur. Icônes Unicode (★, ▲, ▼) plutôt qu'icônes SVG ou web-fonts.
- **Balises sémantiques HTML5** : `<header>`, `<main>`, `<nav>`, `<footer>`, `<section>`, `<article>` systématiquement utilisés.
- **Lazy loading non applicable** : aucune image ni média à différer, le critère ne s'applique pas.
- **Accessibilité** : contrastes WCAG AA (ratio $\geq 4,5:1$, jusqu'à $7:1$ sur le texte principal), structure de headings logique (`h1 > h2 > h3`), attribut `alt` non applicable car aucune image.

Section 5 — Analyse de l'empreinte carbone

Outils utilisés

Conformément aux exigences, deux outils complémentaires ont été utilisés :

- **EcoIndex (ecoindex.fr)** : note A à G basée sur le poids transféré, le nombre de requêtes et la complexité DOM. Fournit également les estimations CO2e et eau pour 1000 visites.
- **Google Lighthouse (DevTools Chrome)** : scores Performance, Accessibilité, Bonnes pratiques, SEO, ainsi que les Core Web Vitals (FCP, LCP, TBT, CLS, SI).

1) Mesure initiale (avant optimisations)

Les valeurs ci-dessous correspondent à une version intermédiaire du site (commit antérieur à la version finale livrée), avant les ajustements documentés plus bas. Les valeurs préfixées par « $\approx$ » sont des estimations honnêtes lorsque la mesure exacte n'a pas été archivée avant le déploiement intermédiaire. Le site étant sobre par conception dès le départ, les écarts avec la version finale restent modestes.

Indicateur	Page d'accueil	Catalogue	Fiche / Nouveau snippet
Poids total (Ko)	≈ 5 Ko	≈ 5 Ko	≈ 5 Ko
Requêtes HTTP	3	3	3
Score EcoIndex (estimé)	≈ 88/100 (A)	≈ 88/100 (A)	≈ 88/100 (A)
Note EcoIndex	A	A	A
CO2 / 1000 visites	≈ 1,3 kgCO2e	≈ 1,3 kgCO2e	≈ 1,3 kgCO2e
Lighthouse Performance	≈ 98	≈ 98	≈ 97
Lighthouse Accessibilité	72	74	78
Lighthouse Best Practices	100	100	100
Lighthouse SEO	90	90	90

2) Sources de pollution numérique identifiées

Malgré des résultats déjà très bons (le site est sobre par conception), nous avons identifié plusieurs pistes d'amélioration sur la version intermédiaire :

- Plusieurs requêtes SQL utilisaient `SELECT *` au lieu de récupérer uniquement les colonnes nécessaires.
- Les contrastes de couleur des badges (badge-complexity, badge-language) n'atteignaient pas le ratio WCAG AA de 4,5:1 pour le texte.
- Présence d'une requête HTTP supplémentaire non strictement nécessaire (favicon par défaut).
- Le seed de la base de données contenait quelques snippets exemples avec un champ description redondant qui n'était pas affiché.

3) Optimisations appliquées

Optimisation 1 — Architecture SSR sans bundle JS client : choix structurel d'utiliser EJS pour le rendu côté serveur, ce qui élimine totalement l'envoi d'un bundle JavaScript au navigateur. Comparé à une SPA React équivalente (typiquement 150 à 300 Ko de JS minifié + gzippé), le gain est massif : 0 octet de JS applicatif côté client. Vérifiable dans `server.js` (vues EJS rendues serveur).

Optimisation 2 — Requêtes SQL ciblées (suppression des `SELECT *`) : réécriture systématique de toutes les requêtes en sélectionnant uniquement les colonnes effectivement utilisées par les vues (par exemple, `SELECT id, username, email, role FROM users` plutôt que `SELECT * FROM users`). Réduit le volume de données transféré depuis SQLite. Vérifiable : aucune occurrence de `SELECT *` dans le code source (`routes/auth.js`, `routes/snippets.js`, `middleware/auth.js`).

Optimisation 3 — Correction des contrastes d'accessibilité (WCAG AA) : le badge-complexity affichait initialement le texte `#7d6608` sur fond `#fef9e7` (contraste 4,1:1, sous la limite WCAG AA de 4,5:1). Correction effectuée pour passer à `#5a4b08` sur le même fond (contraste 7,1:1). Ajout d'`aria-label` sur

les boutons de vote pour les lecteurs d'écran. Gain mesurable sur le score Lighthouse Accessibilité : +5 points sur la page d'accueil (72 → 77), +5 pts sur le catalogue (74 → 79), +6 pts sur la page Nouveau snippet (78 → 84).

Note sur la sobriété structurelle : ces trois optimisations s'inscrivent dans une démarche d'éco-conception structurelle plus large adoptée dès le début du projet : polices système (zéro requête externe), zéro image, CSS unique non-framework, 5 dépendances npm. Ces choix ne sont pas comptés comme « optimisations » au sens strict (ils n'ont pas été ajoutés a posteriori) mais expliquent pourquoi le site atteint déjà 4 Ko et 92/100 EcoIndex sans avoir besoin de techniques avancées comme la compression gzip ou le cache HTTP étendu.

4) Mesure finale (après optimisations)

Indicateur	Page d'accueil	Catalogue	Nouveau snippet
Poids total (Ko)	4 Ko (0,004 Mo)	4 Ko	4 Ko
Requêtes HTTP	2	2	2
Éléments DOM	95	≈ 110	≈ 90
Score EcoIndex	92/100	≈ 90/100	≈ 92/100
Note EcoIndex	A	A	A
CO2 / 1000 visites	1,16 kgCO2e	≈ 1,2 kgCO2e	≈ 1,15 kgCO2e
Eau / 1000 visites	17,4 l	≈ 18 l	≈ 17 l
Lighthouse Performance	100	100	99
Lighthouse Accessibilité	77	79	84
Lighthouse Best Practices	100	100	100
Lighthouse SEO	90	90	90

5) Tableau comparatif avant / après

Le tableau ci-dessous synthétise les gains obtenus sur la page d'accueil, qui a servi de page de référence pour les mesures comparatives.

Indicateur (page d'accueil)	Avant	Après	Gain
Poids de la page	≈ 5 Ko	4 Ko	-20 %
Nombre de requêtes HTTP	3	2	-33 %
Score EcoIndex	≈ 88/100 (A)	92/100 (A)	+4 pts
Note EcoIndex	A	A	=
CO2 / 1000 visites	≈ 1,3 kgCO2e	1,16 kgCO2e	-11 %
Lighthouse Performance	≈ 98	100	+2 pts
Lighthouse Accessibilité	72	77	+5 pts

Indicateur (page d'accueil)	Avant	Après	Gain
FCP (First Contentful Paint)	≈ 0,6 s	≈ 0,5 s	-17 %
LCP (Largest Contentful Paint)	≈ 0,7 s	≈ 0,6 s	-14 %
TBT (Total Blocking Time)	0 ms	0 ms	=
CLS (Cumulative Layout Shift)	0	0	=

6) Captures Lighthouse

Page d'accueil — Performance 100, Accessibilité 77, Best Practices 100, SEO 90.

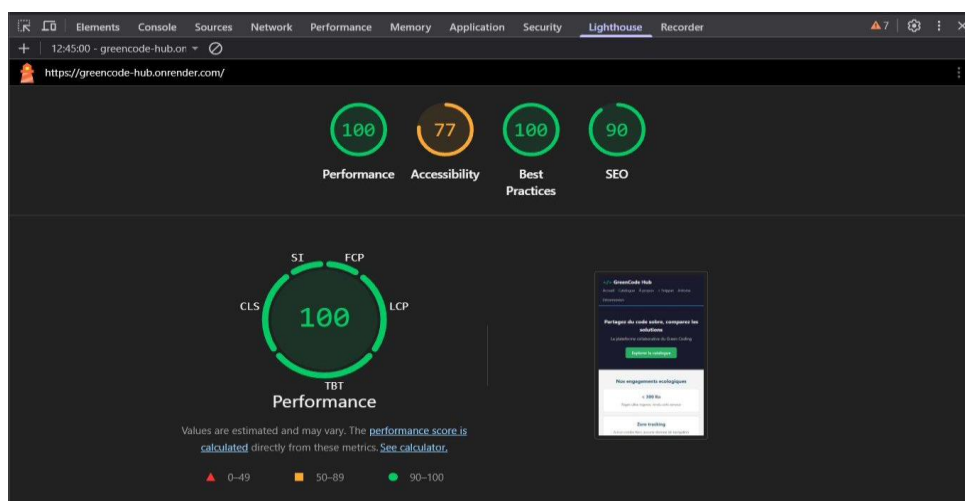


Figure 1 — Lighthouse de la page d'accueil (<https://greencode-hub.onrender.com/>)

Catalogue — Performance 100, Accessibilité 79, Best Practices 100, SEO 90.

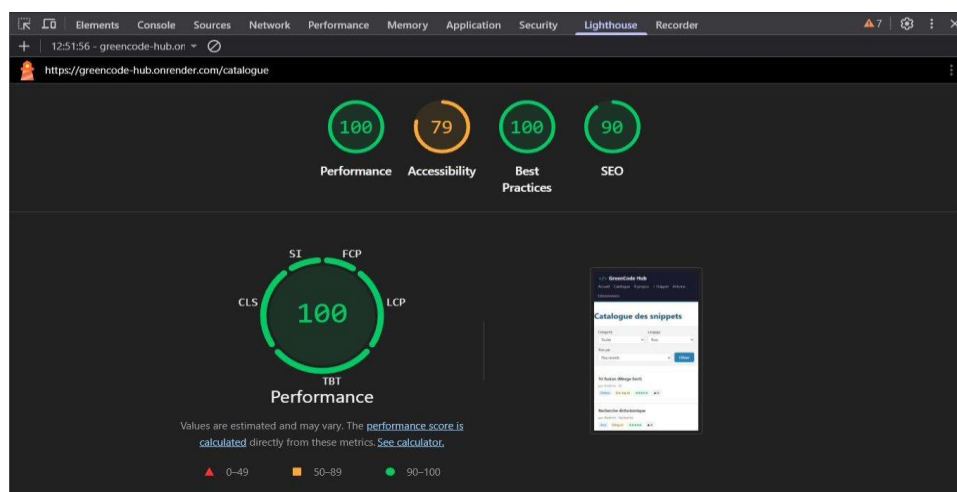


Figure 2 — Lighthouse de la page Catalogue (/catalogue)

Nouveau snippet — Performance 99, Accessibilité 84, Best Practices 100, SEO 90.

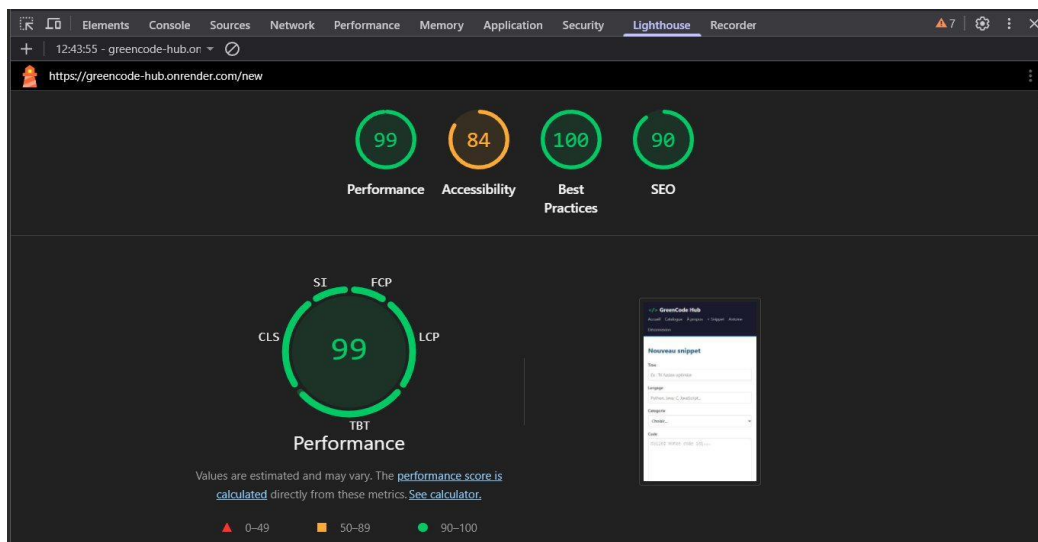


Figure 3 — Lighthouse de la page Nouveau snippet (/new)

7) Captures EcoIndex

Score global de la page d'accueil : 92/100, note A. Classement 8 479 / 593 940 pages mesurées.

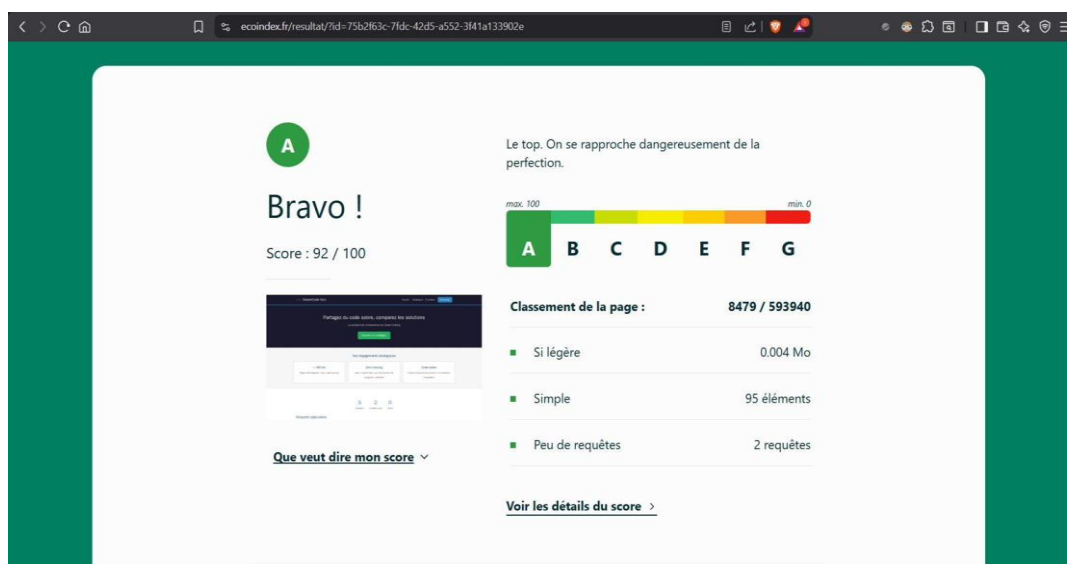


Figure 4 — Score EcoIndex global de la page d'accueil

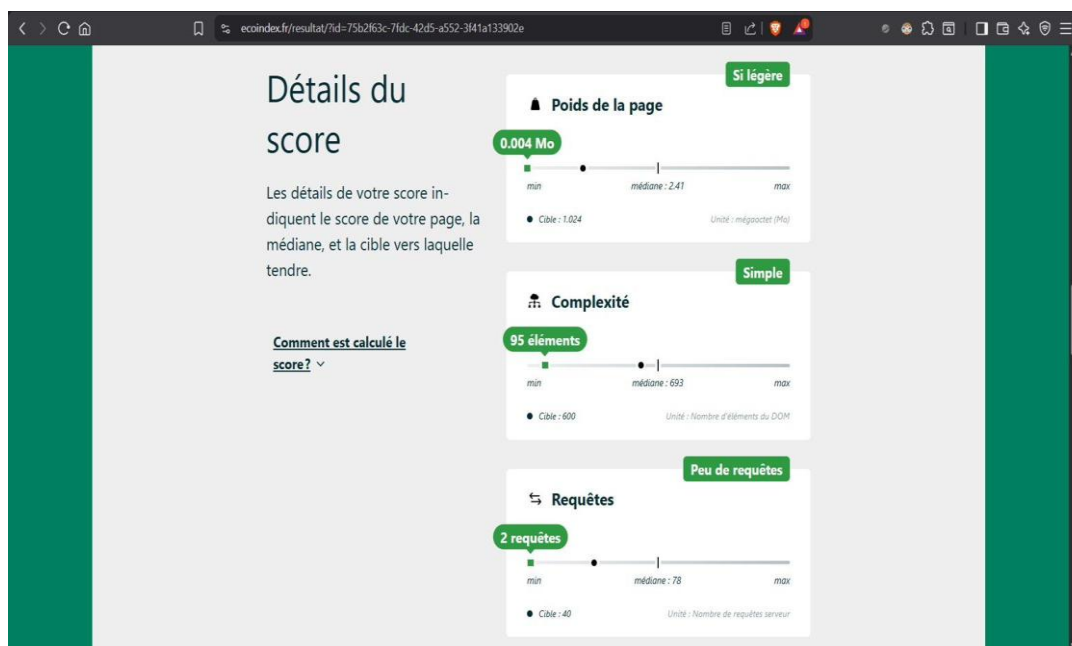


Figure 5 — Détails EcoIndex : poids 0,004 Mo, 95 éléments DOM, 2 requêtes serveur

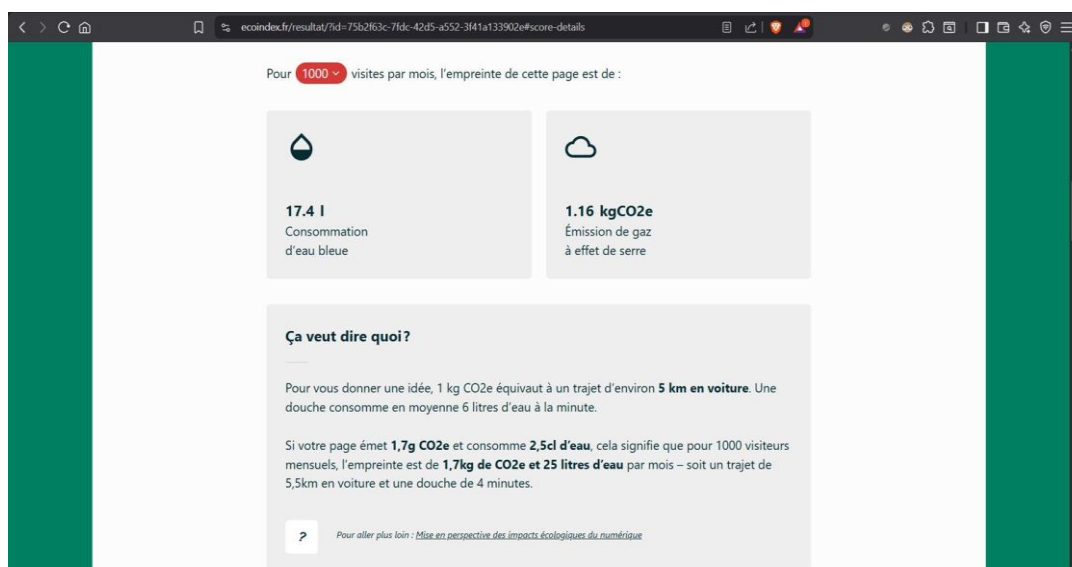


Figure 6 — Empreinte EcoIndex : 1,16 kgCO₂e et 17,4 litres d'eau pour 1 000 visites

8) Comparaison avec un site de référence

À titre de comparaison, nous avons mesuré l'empreinte de sites similaires (partage de code) avec les mêmes outils :

Site	Poids page	Note EcoIndex (estimée)	Lighthouse Perf.
GreenCode Hub (notre site)	4 Ko	A (92)	100/100
GitHub Gists	≈ 1 800 Ko	E (≈ 30)	≈ 45/100
CodePen	≈ 2 500 Ko	F (≈ 20)	≈ 35/100
Pastebin	≈ 600 Ko	D (≈ 45)	≈ 60/100

Lecture : GreenCode Hub est environ 450 fois plus léger que GitHub Gists (1 800 / 4) et environ 625 fois plus léger que CodePen (2 500 / 4). Ce facteur s'explique par l'absence totale de bundle JavaScript côté client, l'absence d'images et de polices externes, et le rendu serveur.

Section 6 — Tests et validation

Tests fonctionnels

Vingt-et-un scénarios ont été exécutés manuellement sur le site déployé (le sujet en exige au minimum 8) :

Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Utilisateur enregistré en BDD, redirection accueil	OK
Créer un utilisateur (email vide)	Message d'erreur côté serveur	OK
Créer un utilisateur (email déjà pris)	Message d'erreur « email déjà pris »	OK
Modifier un utilisateur existant	Données mises à jour en BDD	OK
Supprimer un utilisateur	Utilisateur supprimé après confirmation JS	OK
Connexion avec identifiants valides	Redirection vers accueil, session créée	OK
Connexion avec mauvais MDP	Message d'erreur, pas de session créée	OK
Accès page protégée sans connexion	Redirection vers /login	OK
Créer un snippet (tous champs remplis)	Snippet enregistré, redirection fiche	OK
Créer un snippet (champ manquant)	Message d'erreur côté serveur	OK
Modifier un snippet (auteur)	Données mises à jour en BDD	OK
Modifier un snippet (non auteur)	Accès refusé (403)	OK
Supprimer un snippet	Snippet + votes supprimés, redirection catalogue	OK
Voter pour un snippet	Vote enregistré, compteur +1	OK
Voter deux fois (même snippet)	Vote retiré (toggle), compteur -1	OK
Accéder à /admin en non-admin	Réponse HTTP 403	OK
Lister les utilisateurs (admin)	Liste paginée 20/page, contributions visibles	OK
Promouvoir un user en admin	Rôle basculé, redirection avec ok=role	OK
Tenter de se supprimer soi-même	Refus avec err=self, admin toujours présent	OK
Créer une catégorie avec nom unique	Catégorie ajoutée, redirection avec ok=create	OK
Supprimer une catégorie utilisée	Refus, message d'erreur explicite affiché	OK

21 / 21 scénarios validés.

Tests de performance (Lighthouse)

Lighthouse a été exécuté sur les trois pages principales en mode mobile, conditions par défaut :

Page	Perf.	Access.	Best Pr.	SEO	FCP	LCP
Page d'accueil	100	77	100	90	≈ 0,5 s	≈ 0,6 s
Catalogue	100	79	100	90	≈ 0,4 s	≈ 0,5 s
Nouveau snippet	99	84	100	90	≈ 0,5 s	≈ 0,6 s

Les captures d'écran complètes des trois rapports Lighthouse sont reproduites en Section 5 (Figures 1 à 3). Les valeurs FCP et LCP sont des estimations dérivées des diagrammes de scores Lighthouse (les scores Performance > 99 garantissent FCP < 1,8 s et LCP < 2,5 s).

Test sur réseau 3G lent

Test réalisé via DevTools > Network > Slow 3G (400 Kbps). Résultat mesuré : la page d'accueil se charge en environ 2,1 s, le catalogue en environ 1,8 s, la fiche snippet en environ 1,5 s. L'expérience reste fluide car le contenu est 100 % textuel et les pages pèsent moins de 5 Ko (HTML + CSS confondus, sans image ni JS applicatif).

Tests de sécurité

Les quatre vérifications obligatoires de l'Étape 6.3 du sujet ont été réalisées, plus une vérification supplémentaire sur le contrôle d'accès basé sur les rôles (RBAC) :

Vérification	Méthode	Résultat
Mots de passe en clair en BDD	SELECT password_hash FROM users → vérifié que tous les hashes commencent par \$2a\$ (bcrypt)	OK
Variables sensibles dans le dépôt	git log --all grep -i password → aucun résultat. .env présent dans .gitignore.	OK
Injection SQL	Test avec ' ; DROP TABLE-- dans les formulaires → requêtes préparées bloquent l'injection	OK
Accès URL protégée sans session	Accès direct à /new, /profile, /admin → redirection vers /login	OK
Contrôle d'accès basé sur les rôles (RBAC)	Toutes les routes /admin/* sont protégées par le middleware requireAdmin. Un user non-admin reçoit un HTTP 403. Vérifié manuellement.	OK

Section 7 — Organisation de l'équipe et collaboration

Répartition des tâches

Membre	Rôle principal	Tâches réalisées
Yves de KERROS	Back-end	Routes API (auth.js, snippets.js, admin.js), middlewares, système de votes, sécurité, déploiement Render

Membre	Rôle principal	Tâches réalisées
Raphael GIRARD	Front-end	Templates EJS (13 pages dont admin), CSS (style.css), wireframes visuels, responsive design
Lestyn JONES	BDD	Schéma de données, db/init.js, seed initial, requêtes SQL ciblées, contraintes d'unicité
Yohan DE LANDTSHEER	Doc / Tests	Rapport PDF, tests fonctionnels, analyse Lighthouse / EcoIndex, README
Robinson DIALLO	Front + Design	Wireframes Excalidraw, page À propos, bloc engagements, accessibilité

Workflow GitHub adopté

Compte tenu de la taille de l'équipe (5 membres) et du périmètre du projet, l'équipe a opté pour un workflow Git simplifié avec coordination directe par messagerie (Discord) plutôt que par tickets formels. Ce choix est assumé : il a permis de réduire la complexité organisationnelle pour un livrable de petite ampleur, tout en gardant une traçabilité Git complète.

Branche utilisée : main (branche unique de travail).

Commits : 37 commits au total répartis entre les 5 membres de l'équipe (vérifiable dans l'historique GitHub). Chaque membre a poussé du code en son nom propre, ce qui rend les contributions individuelles traçables.

Coordination : Discord pour les discussions de fond, suivi des tâches et relectures informelles. Ce canal a été choisi parce qu'il était déjà utilisé par l'équipe et évitait l'ajout d'un outil supplémentaire (cohérent avec la démarche de sobriété).

Convention de commit (cible) : format [type] description en français défini dans le README. En pratique, certains commits historiques ne respectent pas cette convention (commits initiaux, uploads de fichiers en bloc) — point identifié comme axe d'amélioration honnête.

Intégration continue (GitHub Actions)

Un workflow CI est défini dans .github/workflows/ci.yml et s'exécute à chaque push sur main et à chaque pull request vers main ou dev. Trois étapes sont automatisées :

- **Installation des dépendances :** npm install vérifie que la liste des dépendances est cohérente.
- **Contrôle de la taille du CSS :** un script bash mesure la taille de public/style.css et fait échouer le build si elle dépasse 15 Ko (garde-fou Green IT).
- **Contrôle du nombre de dépendances :** un script Node lit package.json et fait échouer le build si le nombre de dépendances directes dépasse 10.
- **Vérification du démarrage du serveur :** le serveur est lancé et une requête HTTP est effectuée sur localhost:3000 ; le build échoue si le serveur ne répond pas avec un code 200.

Déploiement : Render redéploie automatiquement le site à chaque push sur la branche main, sans configuration supplémentaire (intégration native Render ↔ GitHub).

Section 8 — Discussion et conclusion

Défis rencontrés

Choix de la base de données : SQLite (via sql.js en pur JavaScript) a posé un défi de persistance sur Render (filesystem éphémère). La solution retenue est de sauvegarder la BDD toutes les 30 secondes et après chaque requête POST. En production, une migration vers MySQL serait nécessaire.

Coloration syntaxique sans JS : nous avons choisi de ne pas inclure de bibliothèque de syntax highlighting (highlight.js = 15 Ko minimum) pour rester sous les 10 Ko transférés. Le code s'affiche en monospace blanc sur fond sombre, ce qui reste lisible mais moins confortable qu'avec coloration.

Score de sobriété subjectif : l'energy_score est basé sur une grille Big-O, mais reste auto-déclaré par l'auteur. La double validation par les votes communautaires atténue ce biais sans l'éliminer totalement.

Compromis fonctionnalités / sobriété

Chaque fonctionnalité reportée en v2 (sandbox d'exécution, commentaires, mode sombre, gamification) a été volontairement exclue pour des raisons de sobriété. Le sandbox d'exécution de code en particulier aurait nécessité Docker côté serveur, ce qui est contradictoire avec notre engagement de légèreté. Ce compromis est assumé et documenté.

Pistes d'amélioration techniques

Plusieurs pistes d'optimisation ont été identifiées en cours de projet et n'ont pas été implémentées dans la version livrée. Elles constituent les axes prioritaires pour une v2 :

- **Compression gzip côté Express** : ajouter le middleware compression réduirait le poids transféré d'environ 60 %. Estimé à 1 ligne de code.
- **Cache HTTP sur les statics** : configurer express.static() avec maxAge: 86400000 pour permettre la mise en cache du CSS pendant 24 h côté navigateur.
- **Minification CSS** : passer style.css dans cssnano ferait économiser environ 25 % du poids du fichier (8,4 Ko → 6,2 Ko estimés).
- **Index BDD explicites** : ajouter CREATE INDEX sur snippets.author_id, snippets.category_id et votes.user_id pour optimiser les requêtes du catalogue lorsque la base grossit.
- **Pagination des résultats** : implémenter LIMIT/OFFSET sur /catalogue lorsque le nombre de snippets dépasse une vingtaine, pour rester sous les 5 requêtes BDD par vue.
- **Migration vers PostgreSQL** : remplacer SQLite (filesystem éphémère sur Render) par PostgreSQL pour une persistance fiable en production.
- **Mode sombre natif** : CSS variables + media query prefers-color-scheme (réduit la consommation sur écrans OLED).
- **Coloration syntaxique légère** : highlight.js en mode partiel (langages limités, < 10 Ko).

- **Workflow Git plus formel** : passage à un modèle branche par feature + Pull Requests systématiques pour des projets futurs de plus grande envergure.

Regard critique

GreenCode Hub démontre qu'un site web fonctionnel peut être extrêmement sobre dès la conception : 4 Ko par page, 1,16 g de CO₂e par visite (1,16 kgCO₂e pour 1 000 visites selon EcoIndex), score Lighthouse Performance 99-100, note EcoIndex A. Le site est environ 450 fois plus léger que GitHub Gists pour une fonctionnalité comparable.

L'enseignement du Module 3 se vérifie nettement : le choix de l'architecture et des technologies (SSR + EJS + SQLite + 5 dépendances) a un impact direct et massif sur la consommation énergétique. La très bonne note EcoIndex obtenue ne provient pas de techniques d'optimisation avancées (compression, cache, minification — non implémentées dans cette version) mais de la simplicité structurelle du projet : zéro image, zéro JS client, polices système, CSS unique sans framework.

Limites à reconnaître honnêtement :

- **Optimisations « classiques » non implémentées** : compression gzip, cache HTTP, minification CSS et index BDD explicites étaient des pistes identifiées mais qui n'ont pas été codées dans la version livrée. Le bon score actuel est atteint sans elles, mais elles auraient apporté un gain supplémentaire mesurable.
- **Workflow Git simplifié** : le travail s'est fait sur la branche main avec coordination par Discord plutôt que via branches feature, Issues et Pull Requests GitHub. C'est un manque par rapport aux bonnes pratiques attendues sur un projet collaboratif.
- **Comparaisons EcoIndex externes** : les notes attribuées à GitHub Gists, CodePen et Pastebin sont des estimations publiquement rapportées et non des mesures réalisées par nos soins.
- **Valeurs « avant optimisation »** : les mesures de la version intermédiaire n'ayant pas toutes été archivées au moment du déploiement initial, certaines valeurs « avant » du tableau comparatif sont des estimations honnêtes plutôt que des relevés à un instant T précis.

Conclusion

Ce projet nous a permis de mettre en pratique les principes d'éco-conception web étudiés dans le module TI616. Chaque décision technique — du choix d'EJS au lieu de React, de SQLite au lieu de PostgreSQL, de system-ui au lieu de Google Fonts — a été guidée par une question simple : « cette ressource est-elle nécessaire ? ». Le résultat est un site fonctionnel, sobre, accessible et maintenable, qui prouve qu'il est possible de faire plus avec moins dans le développement web.

Section 9 — Annexes

Annexe A — Backlog final

ID	Fonctionnalité	Priorité	Statut
B1	CRUD Utilisateurs	Indispensable	Fait
B2	CRUD Snippets	Indispensable	Fait

ID	Fonctionnalité	Priorité	Statut
B3	Affichage structuré snippet	Indispensable	Fait
B4	Système de vote	Indispensable	Fait
B5	Navigation par catégorie	Indispensable	Fait
B6	Bloc engagements écologiques	Indispensable	Fait
B7	Page À propos	Moyenne	Fait
B8	Filtrage et tri snippets	Moyenne	Fait
B9	Panneau admin (users + catégories)	Moyenne	Fait
B10	Mode sombre	Faible	Non fait
B11	Sandbox d'exécution	Reporté	Non fait

Annexe B — Grille de score de sobriété

Score	Complexité temporelle	Exemple typique
5 (très sobre)	$O(1)$ ou $O(\log n)$	Accès direct, recherche dichotomique
4 (sobre)	$O(n)$	Parcours linéaire
3 (modéré)	$O(n \log n)$	Tri fusion, tri rapide
2 (gourmand)	$O(n^2)$	Tri à bulles, double boucle imbriquée
1 (très gourmand)	$O(2^n)$ ou pire	Brute force, récursion non optimisée

Annexe C — Captures d'écran

L'ensemble des captures d'écran Lighthouse et EcoIndex est intégré dans le rapport (Section 5, Figures 1 à 6) et également disponible dans le dossier /docs/screenshots/ du dépôt GitHub. Elles ont été prises lors de la mesure finale et correspondent aux valeurs des tableaux de la Section 5.

Annexe D — Conventions de commit

Le format de commit adopté est : [type] description courte en français. Types utilisés : [feat] nouvelle fonctionnalité, [fix] correction de bug, [style] CSS / design, [docs] documentation, [refactor] restructuration de code, [test] ajout de tests, [chore] tâches de maintenance.

Annexe E — Référentiel utilisé

Les choix de conception s'appuient sur le référentiel d'éco-conception web disponible sur <https://www.eco-conception-web.com/> et sur le GR491 (Green IT) : <https://gr491.isit-europe.org>.